



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in group of two. <table><tr><th>Student 1</th><th>Student 2</th></tr><tr><td>No. 1, 3, 5</td><td>No 2, 4, 6</td></tr></table> Name 1: SYARIFAH DANIA BINTI SYED ABU BAKAR Matric No: A23CS0183 Name 2: AIN NURNABILA BINTI MOHD AZHAR Matric No: A23CS0207	Student 1	Student 2	No. 1, 3, 5	No 2, 4, 6
Student 1	Student 2				
No. 1, 3, 5	No 2, 4, 6				
SUBMISSION DATE:	23 MAY 2024				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

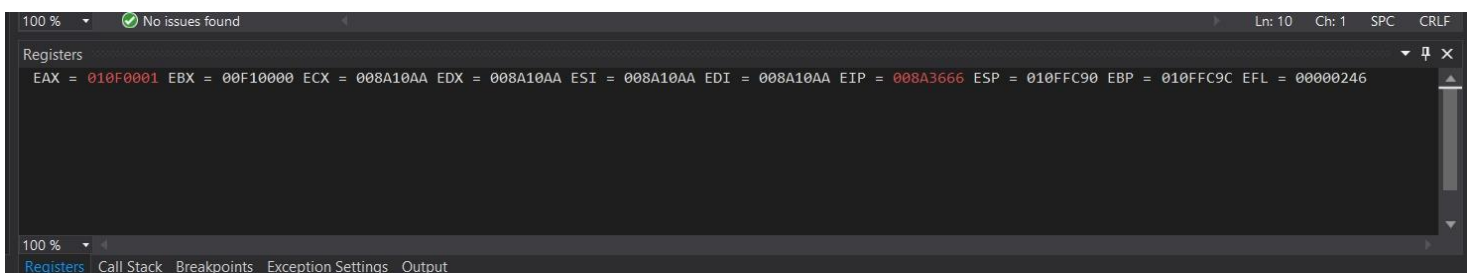
a) Fill (Write) in the contents for the related register in each line:

Table 1

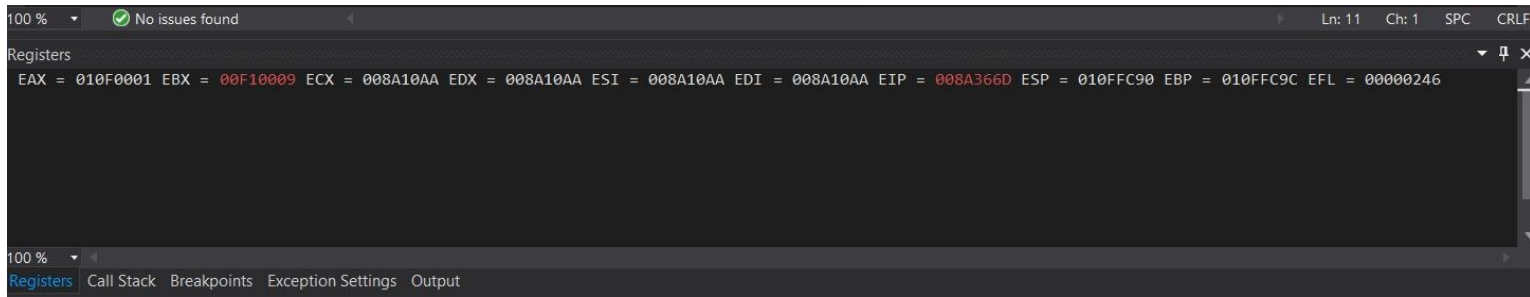
LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	move the value of var2 (9d) into register BX
LINE3	AX = 0009h BX = 0001h	Exchange the value in BX into AX
LINE4	AX = 0009h var1 = 0009h	move the value of AX to var1
LINE5	BX = 0001h var2 = 0001h	move the value of BX to var2

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



LINE2:



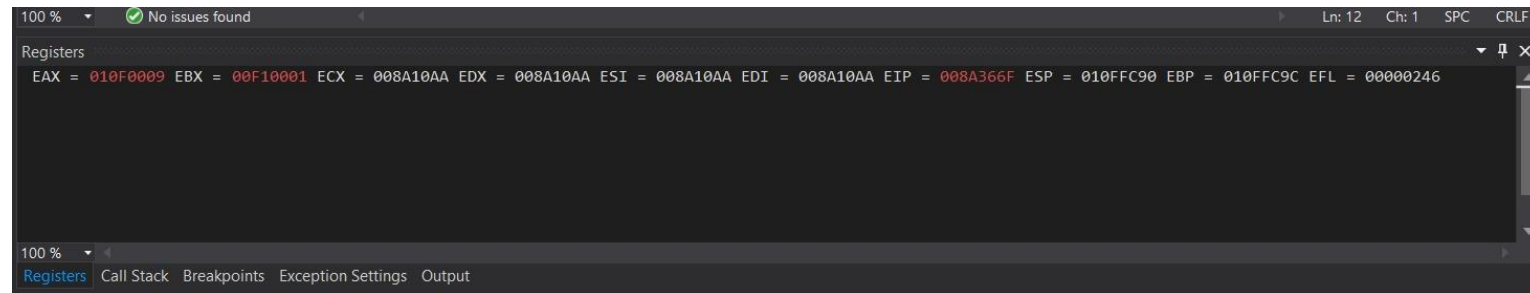
100 % No issues found Ln: 11 Ch: 1 SPC CRLF

Registers

EAX = 010F0001 EBX = 00F10009 ECX = 008A10AA EDX = 008A10AA ESI = 008A10AA EDI = 008A10AA EIP = 008A366D ESP = 010FFC90 EBP = 010FFC9C EFL = 00000246

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE3:



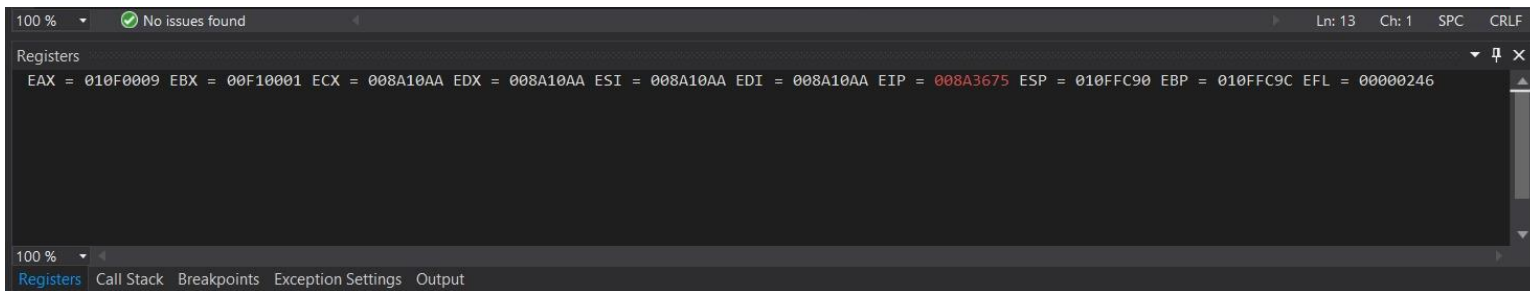
100 % No issues found Ln: 12 Ch: 1 SPC CRLF

Registers

EAX = 010F0009 EBX = 00F10001 ECX = 008A10AA EDX = 008A10AA ESI = 008A10AA EDI = 008A10AA EIP = 008A366F ESP = 010FFC90 EBP = 010FFC9C EFL = 00000246

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE4:



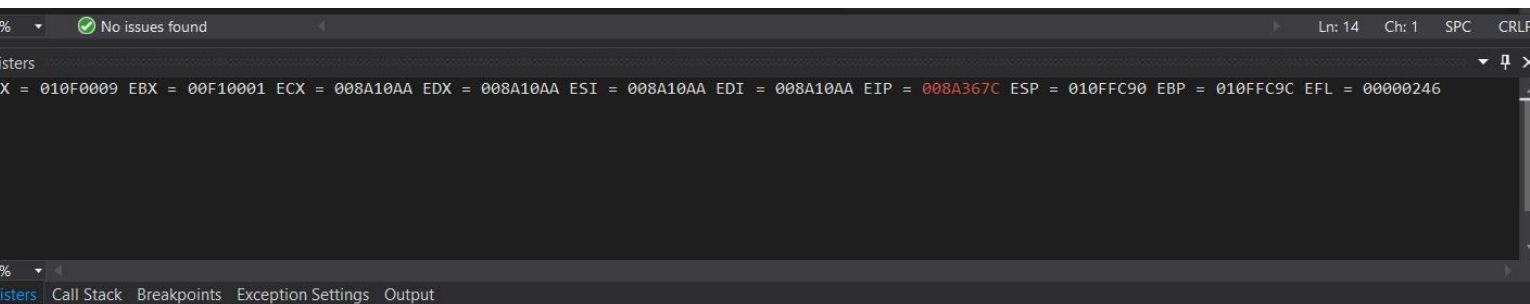
100 % No issues found Ln: 13 Ch: 1 SPC CRLF

Registers

EAX = 010F0009 EBX = 00F10001 ECX = 008A10AA EDX = 008A10AA ESI = 008A10AA EDI = 008A10AA EIP = 008A3675 ESP = 010FFC90 EBP = 010FFC9C EFL = 00000246

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE5:



100 % No issues found Ln: 14 Ch: 1 SPC CRLF

Registers

EAX = 010F0009 EBX = 00F10001 ECX = 008A10AA EDX = 008A10AA ESI = 008A10AA EDI = 008A10AA EIP = 008A367C ESP = 010FFC90 EBP = 010FFC9C EFL = 00000246

100 % Registers Call Stack Breakpoints Exception Settings Output

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval      ; LINE3
    sub ebx,Zval      ; LINE4
    add eax,ebx       ; LINE5
    inc eax           ; LINE6
    mov Rval,eax      ; LINE7
    exit
main endp
end main
```

Answer Q2

a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFEh	Negate the value in EAX (-26)
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX = FFFFFFFFh Zval = 00000028h	Yval is subtracted by Zval (30-40 = -10)
LINE5	EAX = FFFFFFFEh EBX = FFFFFFFFh	Value in EAX register is added by value in EBX (-26 + 10 = -16)
LINE6	EAX = FFFFFFFDh	Value in EAX register is increased by 1 (-16)
LINE7	EAX = FFFFFFFDh Rval = FFFFFFFDh	Value in EAX is moved into Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
100 % 100 % No issues found Ln: 23 Ch: 1 TABS CRLF
Registers
EAX = 0000001A EBX = 00C96000 ECX = 002510AA EDX = 002510AA ESI = 002510AA EDI = 002510AA EIP = 00253665 ESP = 00EFFEE8 EBP = 00EFFEF4 EFL = 00000246

100 %
Registers Call Stack Breakpoints Exception Settings Output
```

LINE2:

```
100 % 100 % No issues found Ln: 25 Ch: 1 TABS CRLF
Registers
EAX = FFFFFFFE EBX = 00C96000 ECX = 002510AA EDX = 002510AA ESI = 002510AA EDI = 002510AA EIP = 00253667 ESP = 00EFFEE8 EBP = 00EFFEF4 EFL = 00000293

0x00256008 = 0000001E

100 %
Registers Call Stack Breakpoints Exception Settings Output
```

LINE3:

```
100 % 100 % No issues found Ln: 27 Ch: 1 TABS CRLF
Registers
EAX = FFFFFFFE EBX = 0000001E ECX = 002510AA EDX = 002510AA ESI = 002510AA EDI = 002510AA EIP = 0025366D ESP = 00EFFEE8 EBP = 00EFFEF4 EFL = 00000293

0x0025600C = 00000028

100 %
Registers Call Stack Breakpoints Exception Settings Output
```

LINE4:

```
100 % 100 % No issues found Ln: 29 Ch: 1 TABS CRLF
Registers
EAX = FFFFFFFE EBX = FFFFFFFF ECX = 002510AA EDX = 002510AA ESI = 002510AA EDI = 002510AA EIP = 00253673 ESP = 00EFFEE8 EBP = 00EFFEF4 EFL = 00000287

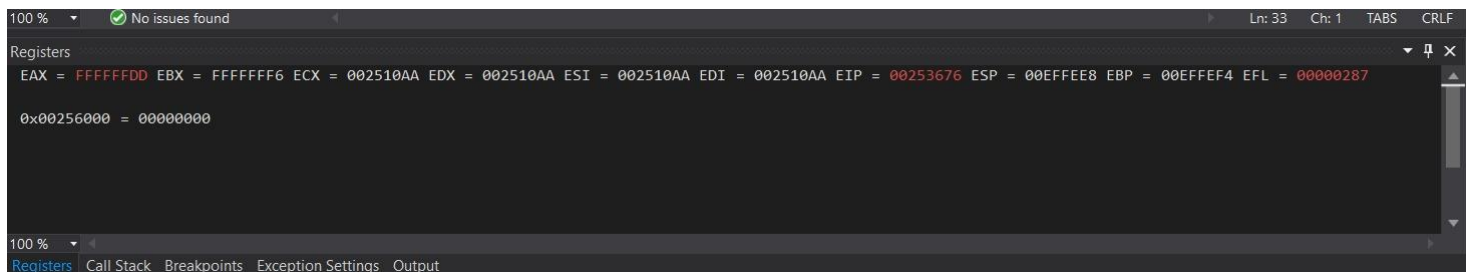
100 %
Registers Call Stack Breakpoints Exception Settings Output
```

LINE5:

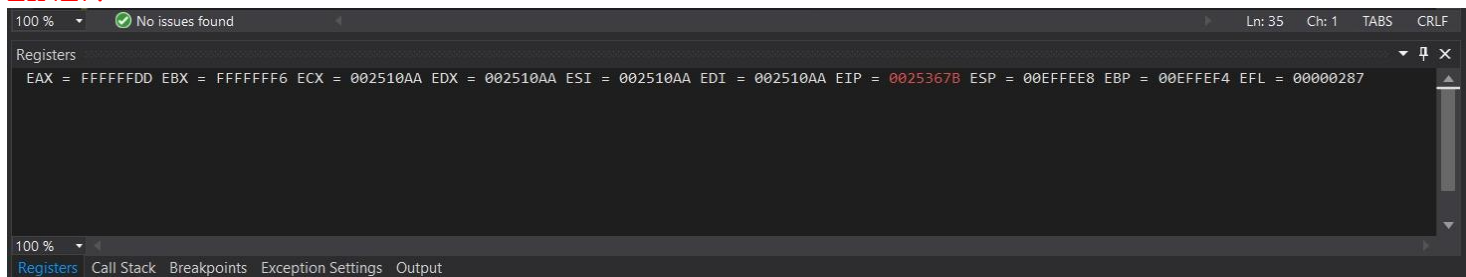
```
100 % 100 % No issues found Ln: 31 Ch: 1 TABS CRLF
Registers
EAX = FFFFFFFD EBX = FFFFFFFF ECX = 002510AA EDX = 002510AA ESI = 002510AA EDI = 002510AA EIP = 00253675 ESP = 00EFFEE8 EBP = 00EFFEF4 EFL = 00000283

100 %
Registers Call Stack Breakpoints Exception Settings Output
```

LINE6:



LINE7:



Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

- Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	Multiply the value of var2 (10d) with EAX
LINE3	EAX = 00000046h var3 = 00000014h	Add the value of var3 (20d) to register EAX
LINE4	EAX = 00000045h var4 = 0	Subtract 1 from register EAX

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

LINE2:

LINE3:

LINE4:

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

Answer Q4

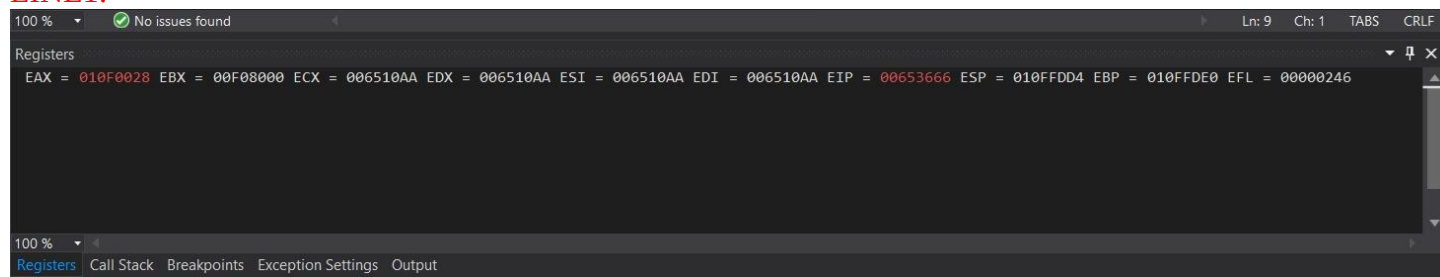
a) Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	move value 5 into register BX
LINE3	AX = 00C8h BX = 0005h	Value in AX is multiplied by value in BX
LINE4	BX = 000Ah var2 = 000Ah	move value var2 (10) into register BX
LINE5	BX = 0007h	Value in register BX is subtracted by 3
LINE6	AX = 001Ch BX = 0007h DX = 0004h	value in register AX is divided by value in register BX (200/7). Current value in AX is quotient which is 1C (28d). Current value in DX is remainder which is 4d.
LINE7	AX = 001Ch var4 = 001Ch	Move value AX (001Ch) into var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:



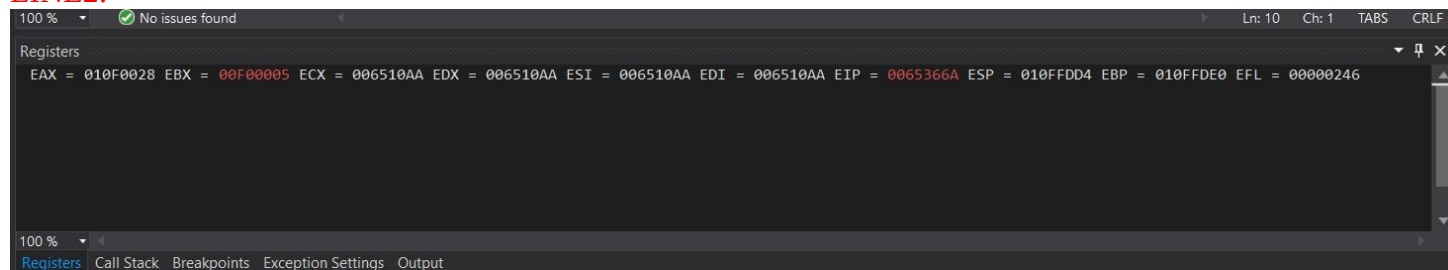
100 % No issues found Ln: 9 Ch: 1 TABS CRLF

Registers

EAX = 010F0028 EBX = 00F08000 ECX = 006510AA EDX = 006510AA ESI = 006510AA EDI = 006510AA EIP = 00653666 ESP = 010FFDD4 EBP = 010FFDE0 EFL = 00000246

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE2:



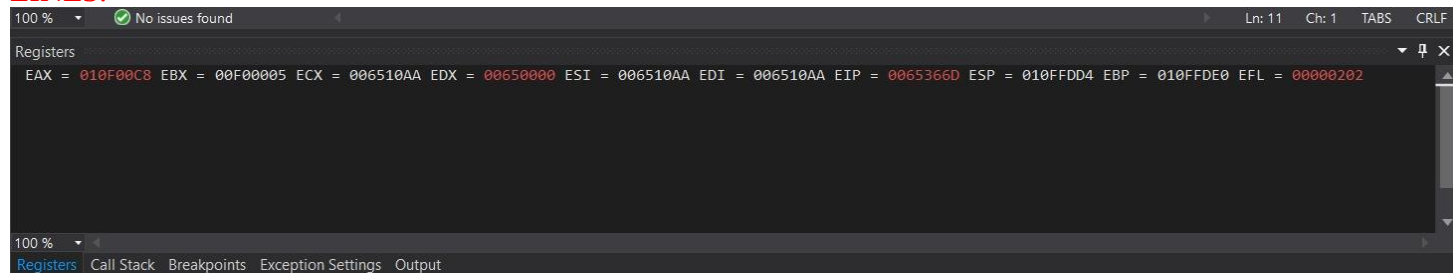
100 % No issues found Ln: 10 Ch: 1 TABS CRLF

Registers

EAX = 010F0028 EBX = 00F00005 ECX = 006510AA EDX = 006510AA ESI = 006510AA EDI = 006510AA EIP = 0065366A ESP = 010FFDD4 EBP = 010FFDE0 EFL = 00000246

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE3:



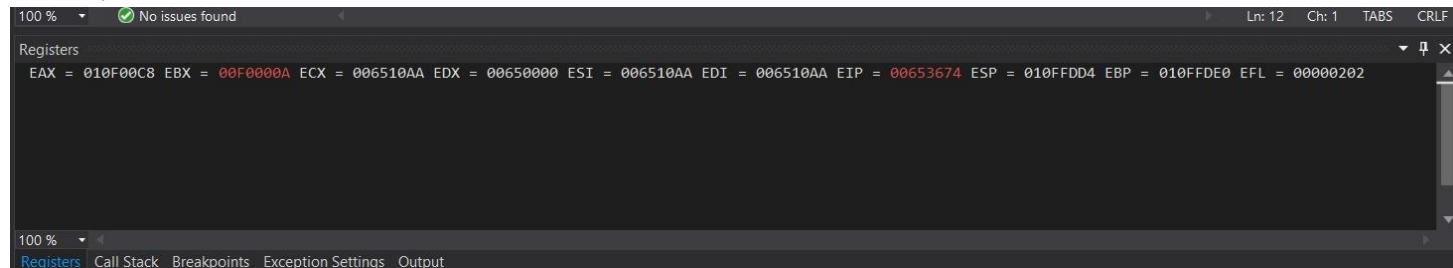
100 % No issues found Ln: 11 Ch: 1 TABS CRLF

Registers

EAX = 010F00C8 EBX = 00F00005 ECX = 006510AA EDX = 00650000 ESI = 006510AA EDI = 006510AA EIP = 0065366D ESP = 010FFDD4 EBP = 010FFDE0 EFL = 00000202

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE4:



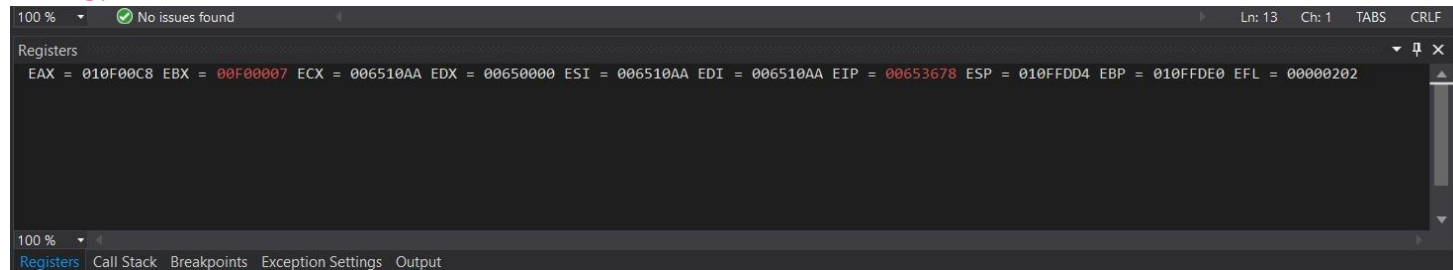
100 % No issues found Ln: 12 Ch: 1 TABS CRLF

Registers

EAX = 010F00C8 EBX = 00F0000A ECX = 006510AA EDX = 00650000 ESI = 006510AA EDI = 006510AA EIP = 00653674 ESP = 010FFDD4 EBP = 010FFDE0 EFL = 00000202

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE5:



100 % No issues found Ln: 13 Ch: 1 TABS CRLF

Registers

EAX = 010F00C8 EBX = 00F00007 ECX = 006510AA EDX = 00650000 ESI = 006510AA EDI = 006510AA EIP = 00653678 ESP = 010FFDD4 EBP = 010FFDE0 EFL = 00000202

100 % Registers Call Stack Breakpoints Exception Settings Output

LINE6:

LINE7:

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
- b. The divisor is specified as the operand of the DIV instruction.
- c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
- d. The remainder is stored in DX.
- e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
- f. If the divisor is 0, a division error occurs.
- g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.

Q5. Given the following instructions as is Code Snippet 1.

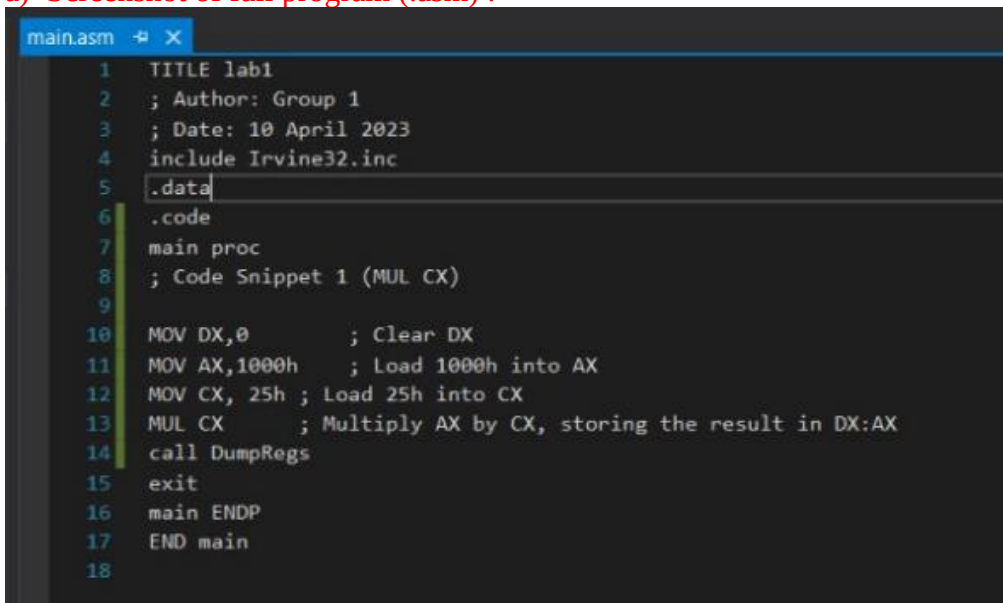
- a) Write a full program to execute the Code Snippet 1.
b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX, 0          ; Clear DX
MOV AX, 1000h       ; Load 1000h into AX
MOV CX, 25h         ; Load 25h into CX
MUL CX              ; Multiply AX by CX, storing the result in DX:AX
```

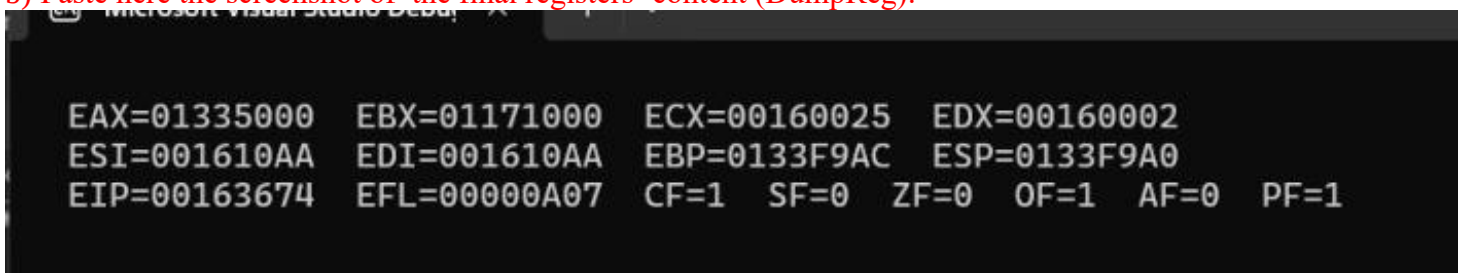
Answer Q5

- a) Screenshot of full program (.asm) :



```
main.asm
1  TITLE lab1
2  ; Author: Group 1
3  ; Date: 10 April 2023
4  include Irvine32.inc
5  .data
6  .code
7  main proc
8  ; Code Snippet 1 (MUL CX)
9
10 MOV DX, 0          ; Clear DX
11 MOV AX, 1000h       ; Load 1000h into AX
12 MOV CX, 25h         ; Load 25h into CX
13 MUL CX              ; Multiply AX by CX, storing the result in DX:AX
14 call DumpRegs
15 exit
16 main ENDP
17 END main
18
```

- b) Paste here the screenshot of the final registers' content (DumpReg):



```
EAX=01335000  EBX=01171000  ECX=00160025  EDX=00160002
ESI=001610AA  EDI=001610AA  EBP=0133F9AC  ESP=0133F9A0
EIP=00163674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1
```

Q6. Given the following instructions as is Code Snippet 2.

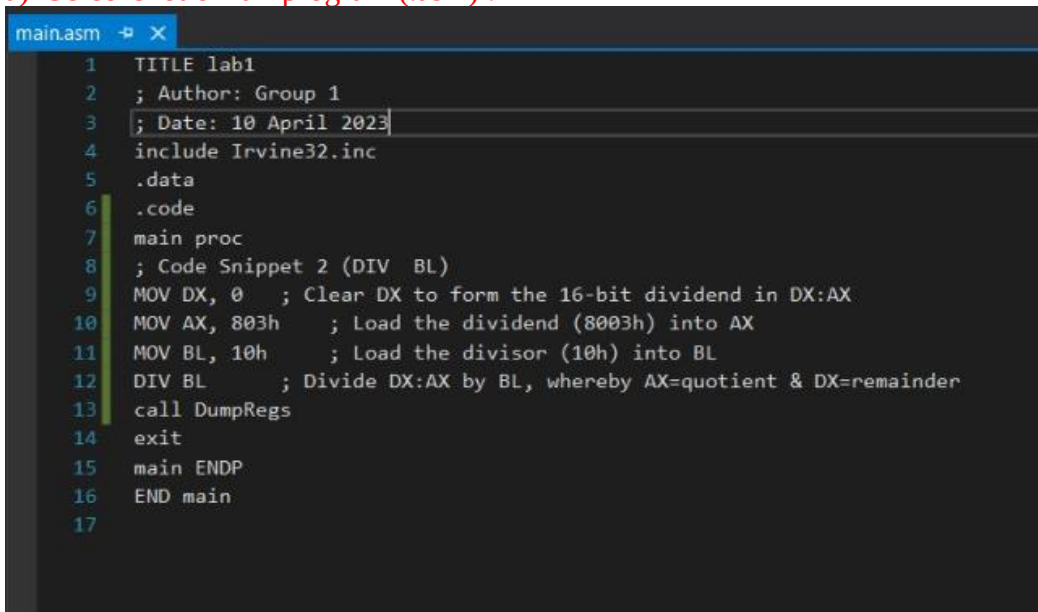
- a) Write a full program to execute the Code Snippet 2.
b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0          ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h        ; Load the dividend (8003h) into AX
MOV BL, 10h         ; Load the divisor (10h) into BL
DIV BL              ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

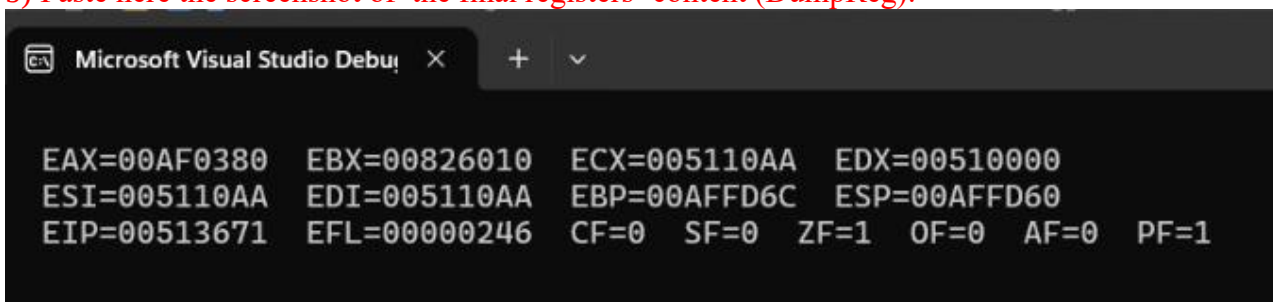
Answer Q6

a) Screenshot of full program (.asm) :

A screenshot of a text editor window titled 'main.asm'. The code is an x86 assembly program. It starts with a title 'lab1', author 'Group 1', and date '10 April 2023'. It includes 'Irvine32.inc'. The program is in the 'main' procedure. It sets up a 16-bit dividend in DX:AX by clearing DX and loading 8003h into AX. It loads the divisor 10h into BL. It then performs a 16-bit division (DIV BL), which sets the quotient in AX and the remainder in DX. Finally, it calls 'DumpRegs' to display the register contents, exits the procedure, and ends the main program.

```
1  TITLE lab1
2  ; Author: Group 1
3  ; Date: 10 April 2023
4  include Irvine32.inc
5  .data
6  .code
7  main proc
8  ; Code Snippet 2 (DIV BL)
9  MOV DX, 0    ; Clear DX to form the 16-bit dividend in DX:AX
10 MOV AX, 803h  ; Load the dividend (8003h) into AX
11 MOV BL, 10h   ; Load the divisor (10h) into BL
12 DIV BL        ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
13 call DumpRegs
14 exit
15 main ENDP
16 END main
17
```

b) Paste here the screenshot of the final registers' content (DumpReg):

A screenshot of the 'DumpRegs' function output in a debugger window titled 'Microsoft Visual Studio Debug Console'. It displays the final values of the x86 registers and status flags after the division operation. The registers are shown in four columns: EAX, EBX, ECX, EDX; ESI, EDI, EBP, ESP; and EIP, EFL. The status flags CF, SF, ZF, OF, AF, and PF are also shown.

```
Microsoft Visual Studio Debug Console
EAX=00AF0380  EBX=00826010  ECX=005110AA  EDX=00510000
ESI=005110AA  EDI=005110AA  EBP=00AFFD6C  ESP=00AFFD60
EIP=00513671  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1
```

YOUTUBE VIDEO LINK:

<https://youtu.be/HYnG1VsMC8U>